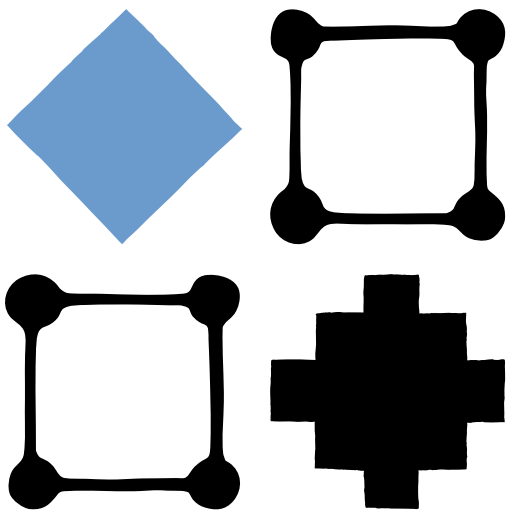


Engineering at Anthropic

Anthropic 的工程



Harness design for long-running application development 用于长期应用开发的线束设计

Harness design is key to performance at the frontier of agentic coding. Here's how we pushed Claude further in frontend design and long-running autonomous software engineering.

线束设计是代理编码前沿性能的关键。以下是我们如何推动 Claude 在前端设计和长期自主软件工程领域更进一步。

Published Mar 24, 2026 发表于2026年3月24日

Written by Prithvi Rajasekaran, a member of our Labs team.

本文作者为我们实验室团队成员 Prithvi Rajasekaran。

Over the past several months I've been working on two interconnected problems: getting Claude to produce high-quality frontend designs, and getting it to build complete applications without human intervention. This work originated with earlier efforts on our frontend design skill and long-running coding agent harness, where my colleagues and I were able to improve Claude's performance well above baseline through prompt engineering and harness design—but both eventually hit ceilings.

过去几个月里，我一直在处理两个相互关联的问题：让 Claude 能够生成高质量的前端设计，以及让它在无需人工干预的情况下构建完整应用程序。这项工作起源于我们早期在前端设计技能和长期运行的编码代理工具框架上的努力，我和同事们通过提示工程和工具束设计，将 Claude 的性能提升远超基线——但最终两者都达到了瓶颈。

To break through, I sought out novel AI engineering approaches that held across two quite different domains, one defined by subjective taste, the other by verifiable correctness and usability. Taking inspiration from Generative Adversarial Networks (GANs), I designed a multi-agent structure with a **generator** and **evaluator** agent. Building an evaluator that graded outputs reliably—and with taste—meant first developing a set of criteria that could turn subjective judgments like “is this design good?” into concrete, gradable terms.

为了突破，我寻求了跨越两个截然不同领域的新型 AI 工程方法，一个由主观品味定义，另一个以可验证的正确性和可用性为定义。我从生成对抗网络（GAN）中汲取灵感，设计了一个多智能体结构，包含生成器和评估代理。构建一个可靠且符合个人口味的评估器，意味着首先开发一套标准，将主观判断如“这个设计好吗？”转化为具体且可评分的术语。

I then applied these techniques to long-running autonomous coding, carrying over two lessons from our earlier harness work: decomposing the build into tractable chunks, and using structured artifacts to hand off context between sessions. The final result was a three-agent architecture—planner, generator, and evaluator—that produced rich full-stack applications over multi-hour autonomous coding sessions.

随后，我将这些技术应用于长期运行的自主编码，并从早期的约束工具工作中汲取了两个经验：将构建分解为可处理的块，以及利用结构化工件在会话间传递上下文。最终成果是一个三智能体架构——规划器、生成器和评估者——通过数小时的自主编码会话生成丰富的全栈应用。

Why naive implementations fall short

为什么朴素的实现会失败

We've previously shown that harness design has a substantial impact on the effectiveness of long running agentic coding. In an earlier experiment, we used an initializer agent to decompose a product spec into a task list, and a coding agent that implemented the tasks one feature at a time before handing off artifacts to carry context across sessions. The broader developer community has converged on similar insights, with approaches like the "Ralph Wiggum" method using hooks or scripts to keep agents in continuous iteration cycles.

我们此前已证明，束带设计对长期智能体编码的有效性有显著影响。在早期的实验中，我们使用初始化代理将产品规格分解为任务列表，并使用编码代理逐个功能实现任务，然后将工件交接到会话间传递上下文。更广泛的开发者社区也达成了类似的见解，采用了“Ralph Wiggum”方法，利用钩子或脚本保持代理处于连续迭代周期。

But some problems remained persistent. For more complex tasks, the agent still tends to go off the rails over time. While decomposing this issue, we observed two common failure modes with agents executing these sorts of tasks.

但一些问题依然存在。对于更复杂的任务，代理仍然容易随着时间推移偏离轨道。在分析这个问题时，我们观察到执行此类任务时有两种常见的失败模式。

First is that models tend to lose coherence on lengthy tasks as the context window fills (see our post on context engineering). Some models also exhibit "context anxiety," in which they begin wrapping up work prematurely as they approach what they believe is their context limit. Context resets—clearing the context window entirely and starting a fresh agent, combined with a structured handoff that carries the previous agent's state and the next steps—addresses both these issues.

首先，模型在长时间任务中随着上下文窗口的填满，往往会失去连贯性（参见我们关于上下文工程的文章）。一些模型还表现出“上下文焦虑”，即在接近他们认为的上下文极限时，开始过早结束工作。上下文重置——完全清除上下文窗口并重新启动一个代理，结合结构化的切换，传递前一个代理的状态和下一步——解决了这两个问题。

This differs from compaction, where earlier parts of the conversation are summarized in place so the same agent can keep going on a shortened history. While compaction preserves continuity, it doesn't give the agent a clean slate, which means context anxiety can still persist. A reset provides a clean slate, at the cost of the handoff artifact having enough state for the next agent to pick up the work cleanly. In our earlier testing, we found Claude Sonnet 4.5 exhibited context anxiety strongly enough that compaction alone wasn't sufficient to enable strong long task performance, so context resets became essential to the harness design. This solves the core issue, but adds orchestration complexity, token overhead, and

latency to each harness run.

这与压缩不同，压缩中对话的早期部分被总结，以便同一代理能在缩短的历史中继续。虽然压缩保持了连续性，但不会给代理一个干净的起点，这意味着上下文焦虑仍然可能存在。重置则提供一个干净的起点，但代价是切换伪造物有足够的状态，让下一个代理能够干净地接手工作。在我们早期的测试中，我们发现 Claude Sonnet 4.5 表现出强烈的上下文焦虑，仅靠压缩不足以实现强劲的长任务表现，因此上下文重置成为线束设计的必不可少。这解决了核心问题，但为每次线束运行增加了编排复杂度、令牌开销和延迟。

A second issue, which we haven't previously addressed, is self-evaluation. When asked to evaluate work they've produced, agents tend to respond by confidently praising the work—even when, to a human observer, the quality is obviously mediocre. This problem is particularly pronounced for subjective tasks like design, where there is no binary check equivalent to a verifiable software test. Whether a layout feels polished or generic is a judgment call, and agents reliably skew positive when grading their own work.

第二个问题，我们之前未涉及过，是自我评价。当被要求评价自己制作的作品时，代理往往自信地赞扬作品——即使对人类观察者来说，质量显然平庸。这个问题在主观任务如设计中尤为明显，因为没有可验证的软件测试的二元检查。版面是否显得精致或通用是判断问题，代理在评分自己作品时通常会偏向正面。

However, even on tasks that do have verifiable outcomes, agents still sometimes exhibit poor judgment that impedes their performance while completing the task. Separating the agent doing the work from the agent judging it proves to be a strong lever to address this issue. The separation doesn't immediately eliminate that leniency on its own; the evaluator is still an LLM that is inclined to be generous towards LLM-generated outputs. But tuning a standalone evaluator to be skeptical turns out to be far more tractable than making a generator critical of its own work, and once that external feedback exists, the generator has something concrete to iterate against.

然而，即使在有可验证结果的任务上，代理有时仍会表现出判断力差，影响完成任务的表现。将执行工作的代理与判断代理区分开来，成为解决这一问题的有力杠杆。分离本身并不能立即消除这种宽容；评估者仍是一个倾向于对大型语言模型生成输出宽容的大型语言模型。但将独立评估者调校成怀疑者，比让生成器批评自身工作要容易得多，一旦外部反馈存在，生成器就有了具体的反复试验。

Frontend design: making subjective quality gradable

前端设计：使主观质量可评分

I started by experimenting on frontend design, where the self-evaluation issue was most visible. Absent any intervention, Claude normally gravitates toward safe, predictable layouts that are technically functional but visually unremarkable.

我从前端设计开始尝试，这里的自我评估问题最为明显。在没有任何干预的情况下，Claude 通常倾向于安全、可预测的布局，这些布局技术上功能正常但视觉上并不突出。

Two insights shaped the harness I built for frontend design. First, while aesthetics can't be fully reduced to a score—and individual tastes will always vary—they can be improved with grading criteria that encode design principles and preferences. "Is this design beautiful?" is hard to answer consistently, but "does this follow our principles for good design?" gives Claude something concrete to grade against. Second, by separating frontend generation from frontend grading, we can create a feedback loop that drives the generator toward stronger outputs.

有两个洞见塑造了我为前端设计构建的框架。首先，虽然美学无法完全归结为评分——每个人的喜好总是不同——但通过编码设计原则和偏好的评分标准，它们可以得到改进。“这个设计漂亮吗？”很难一贯地回答，但“这是否符合我们良好设计的原则？”给了 Claude 一个具体的评分依据。其次，通过将前端生成与前端评分分离，我们可以创造一个反馈循环，推动生成器朝着更强的输出方向发展。

With this in mind, I wrote four grading criteria that I gave to both the generator and evaluator agents in their prompts:

考虑到这一点，我写下了四个评分标准，分别在他们的提示中给出了生成器和评估代理：

- **Design quality:** Does the design feel like a coherent whole rather than a collection of parts? Strong work here means the colors, typography, layout, imagery, and other details combine to create a distinct mood and identity.

设计质量：设计感觉像是一个连贯的整体，而不是部分的集合？这里的优秀作品意味着色彩、排版、布局、图像及其他细节结合起来，营造出独特的氛围和身份。

- **Originality:** Is there evidence of custom decisions, or is this template layouts, library defaults, and AI-generated patterns? A human designer should recognize deliberate creative choices. Unmodified stock components—or telltale signs of AI generation like purple gradients over white cards—fail here.

原创性：有没有自定义决策的证据，还是这些只是模板布局、库默认和 AI 生成的图案？人类设计师应当识别有意为之的创意选择。未经修改的原厂组件——或者像白色卡片上的紫色渐变等 AI 生成的明显迹象——在这里都失效了。

- **Craft:** Technical execution: typography hierarchy, spacing consistency, color harmony, contrast ratios. This is a competence check rather than a creativity check. Most reasonable implementations do fine here by default; failing means broken fundamentals.

工艺：技术执行：排版层级、间距一致性、色彩和谐、对比度。这是能力检定，而不是创造力检定。大多数合理的实现默认情况下都能很好地实现；失败意味着基本功破碎。

- **Functionality:** Usability independent of aesthetics. Can users understand what the interface does, find primary actions, and complete tasks without guessing?

功能：可用性独立于美观。用户能否理解界面的作用，找到主要操作，并完成任务而不猜测？

I emphasized design quality and originality over craft and functionality. Claude already scored well on craft and functionality by default, as the required technical competence tended to come naturally to the model. But on design and originality, Claude often produced outputs that were bland at best. The criteria explicitly penalized highly generic “AI slop” patterns, and by weighting design and originality more heavily it pushed the model toward more aesthetic risk-taking.

我强调设计质量和原创性，而非工艺和功能性。Claude 在工艺和功能性方面默认就得分不错，因为所需的技术能力对模型来说是自然而然的。但在设计和原创性方面，克劳德常常产出的作品充其量也只是平淡无奇。这些标准明确惩罚了高度通用的“AI 杂质”模式，并通过更重视设计和原创性，推动模型趋向于更具美学风险的方向。

I calibrated the evaluator using few-shot examples with detailed score breakdowns. This ensured the evaluator’s judgment aligned with my preferences, and reduced score drift across iterations.

我用少量样本和详细的分数分析来校准评估器。这确保了评估者的判断与我的偏好一致，减少了多次迭代间的分数漂移。

I built the loop on the Claude Agent SDK, which kept the orchestration straightforward. A generator agent first created an HTML/CSS/JS frontend based on a user prompt. I gave the evaluator the Playwright MCP, which let it interact with the live page directly before scoring each criterion and writing a detailed critique. In practice, the evaluator would navigate the page on its own, screenshotting and carefully studying the implementation before producing its assessment. That feedback flowed back to the generator as input for the next iteration. I ran 5 to 15 iterations per generation, with each iteration typically pushing the generator in a more distinctive direction as it responded to the evaluator's critique. Because the evaluator was actively navigating the page rather

than scoring a static screenshot, each cycle took real wall-clock time. Full runs stretched up to four hours. I also instructed the generator to make a strategic decision after each evaluation: refine the current direction if scores were trending well, or pivot to an entirely different aesthetic if the approach wasn't working.

我用 Claude Agent SDK 构建了循环，它让编排变得简单明了。生成器代理首先根据用户提示创建了 HTML/CSS/JS 前端。我给评审员提供了剧作家 MCP，让它能在评分每个标准和写详细批评前直接与实时页面互动。实际上，评估者会自行浏览页面，截图并仔细研究实现过程，然后再做出评估。这些反馈到生成器，作为下一次迭代的输入。我每代运行 5 到 15 次迭代，每次迭代通常都会推动生成器朝更独特的方向发展，以响应评估者的批评。因为评审员是主动浏览页面，而不是用静态截图评分，每个周期都耗费了大量的墙上时间。完整演出时间长达四小时。我还指示生成器在每次评估后做出战略决策：如果分数趋势良好，就调整当前方向；如果方法不奏效，则转向完全不同的美学。

Across runs, the evaluator's assessments improved over iterations before plateauing, with headroom still remaining. Some generations refined incrementally. Others took sharp aesthetic turns between iterations.

在多次运行中，评估者的评估在多次迭代中有所提升，但最终趋于平稳，且头间余量仍然存在。有些世代则是逐步精炼的。另一些则在迭代间发生了尖锐的美学转变。

The wording of the criteria steered the generator in ways I didn't fully anticipate. Including phrases like "the best designs are museum quality" pushed designs toward a particular visual convergence, suggesting that the prompting associated with the criteria directly shaped the character of the output.

这些标准的措辞以我未曾完全预料的方式引导了生成器的发展。包含“最佳设计是博物馆级”等短语推动设计趋向特定的视觉趋同，表明与标准相关的提示直接塑造了成果的性质。

While scores generally improved over iterations, the pattern was not always cleanly linear. Later implementations tended to be better as a whole, but I regularly saw cases where I preferred a middle iteration over the last one. Implementation complexity also tended to increase across rounds, with the generator reaching for more ambitious solutions in response to the evaluator's feedback. Even on the first iteration, outputs were noticeably better than a baseline with no prompting at all, suggesting the criteria and associated language themselves steered the model away from generic defaults before any evaluator feedback led to further

refinement.

虽然分数通常会随着迭代而提升，但这种模式并不总是干净利落的线性。后续实现整体上通常更好，但我经常看到我更喜欢中间版本而非上一个版本的情况。实施复杂度也随着评估者反馈而增加，生成器会根据评估者的反馈寻求更具雄心的解决方案。即使在第一次迭代中，输出也明显优于无提示的基线，表明标准和相关语言本身引导模型远离了通用默认，直到评估者反馈促使进一步完善。

In one notable example, I prompted the model to create a website for a Dutch art museum. By the ninth iteration, it had produced a clean, dark-themed landing page for a fictional museum. The page was visually polished but largely in line with my expectations. Then, on the tenth cycle, it scrapped the approach entirely and reimagined the site as a spatial experience: a 3D room with a checkered floor rendered in CSS perspective, artwork hung on the walls in free-form positions, and doorway-based navigation between gallery rooms instead of scroll or click. It was the kind of creative leap that I hadn't seen before from a single-pass generation.

有一个显著例子，我让模特为荷兰一家艺术博物馆创建了一个网站。到了第九次，它已经制作了一个干净、暗色调的虚构博物馆登陆页。页面视觉上打磨得很精致，但基本符合我的预期。随后，在第十个周期中，它完全放弃了这种做法，将网站重新构想为一种空间体验：一个 3D 房间，地板为棋盘格，采用 CSS 透视，墙上自由悬挂艺术品，画廊间的门口导航取代滚动或点击。这是我以前从一代人中从未见过的创造飞跃。

0:00 / 0:28

Scaling to full-stack coding

向全栈编码扩展

With these findings in hand, I applied this GAN-inspired pattern to full-stack development. The generator-evaluator loop maps naturally onto the software development lifecycle, where code review and QA serve the same structural role as the design evaluator.

基于这些发现，我将这一受 GAN 启发的模式应用于全栈开发。生成器-评估者循环自然地映射到软件开发生命周期，在其中代码审查和质量保证承担与设计评估者相同的结构性角色。

The architecture 建筑风格

In our earlier long-running harness, we had solved for coherent multi-session coding with an initializer agent, a coding agent that worked one feature at a time, and context resets between sessions. Context resets were a key unlock: the harness used Sonnet 4.5, which exhibited the “context anxiety” tendency mentioned earlier. Creating a harness that worked well across context resets was key to keeping the model on task. Opus 4.5 largely removed that behavior on its own, so I was able to drop context resets from this harness entirely. The agents were run as one continuous session across the whole build, with the Claude Agent SDK's automatic compaction handling context growth along the way.

在我们早期的长期运行框架中，我们通过初始化代理、一个一次处理一个功能、会话间的上下文重置来解决连贯多会话编码的问题。上下文重置是关键：该束带使用了 Sonnet 4.5，表现出前述的“上下文焦虑”倾向。创建一个能够在上下文重置中良好运行的束缚，是保持模型专注任务的关键。Opus 4.5 基本上自己消除了这种行为，所以我可以完全从这个接口中移除上下文重置。这些代理在整个构建中作为一个连续会话运行，Claude Agent SDK 的自动压缩处理上下文增长也随之推进。

For this work I built on the foundation from the original harness with a three-agent system, with each agent addressing a specific gap I'd observed in prior runs. The system contained the following agent personas:

这次工作中，我基于原始线束建立了三代理系统，每个代理分别针对我之前观察到的特定漏洞进行补偿。该系统包含以下代理角色：

Planner: Our previous long-running harness required the user to provide a detailed spec upfront. I wanted to automate that step, so I created a planner agent that took a simple 1-4 sentence prompt and expanded it into a full product spec. I prompted it to be ambitious about scope and to stay focused on product context

and high level technical design rather than detailed technical implementation. This emphasis was due to the concern that if the planner tried to specify granular technical details upfront and got something wrong, the errors in the spec would cascade into the downstream implementation. It seemed smarter to constrain the agents on the deliverables to be produced and let them figure out the path as they worked. I also asked the planner to find opportunities to weave AI features into the product specs. (See example in the Appendix at the bottom.)

规划师： 我们之前的长期使用背带要求用户事先提供详细规格。我想自动化这一步，于是我创建了一个规划代理，将一个简单的 1-4 句话提示扩展成完整的产品规格。我建议它对范围有雄心，专注于产品背景和高层技术设计，而非详细的技术实现。这种强调源于担心如果规划者试图提前明确细致的技术细节却出错，规范中的错误会在后续实施中连锁反应。似乎更聪明的做法是限制特工们交付的成果，让他们边工作边摸索路径。我还要求策划者寻找将 AI 功能融入产品规格的机会。（详见底部附录中的示例。）

Generator: The one-feature-at-a-time approach from the earlier harness worked well for scope management. I applied a similar model here, instructing the generator to work in sprints, picking up one feature at a time from the spec. Each sprint implemented the app with a React, Vite, FastAPI, and SQLite (later PostgreSQL) stack, and the generator was instructed to self-evaluate its work at the end of each sprint before handing off to QA. It also had git for version control.

生成器： 早期框架中一次只做一个功能的方式在范围管理中效果很好。我在这里应用了类似的模型，指示生成器在冲刺中工作，一次从规格中获取一个特征。每个冲刺都用 React、Vite、FastAPI 和 SQLite（后来的 PostgreSQL）协议栈实现了应用，生成器被指示在每个冲刺结束时自我评估其工作后再交给质量保证。它还支持了 git 作为版本控制。

Evaluator: Applications from earlier harnesses often looked impressive but still had real bugs when you actually tried to use them. To catch these, the evaluator used the Playwright MCP to click through the running application the way a user would, testing UI features, API endpoints, and database states. It then graded each sprint against both the bugs it had found and a set of criteria modeled on the frontend experiment, adapted here to cover product depth, functionality, visual design, and code quality. Each criterion had a hard threshold, and if any one fell below it, the sprint failed and the generator got detailed feedback on what went wrong.

评估员：早期线束的应用看起来很出色，但实际使用时仍存在真实的 bug。为了捕捉这些问题，评估者使用 Playwright MCP 像用户一样点击运行中的应用程序，测试 UI 功能、API 端点和数据库状态。然后它根据发现的漏洞和基于前端实验建模的一套标准对每个冲刺进行评分，这些标准在这里调整以涵盖产品深度、功能性、视觉设计和代码质量。每个标准都有硬性门槛，如果有一项低于阈值，冲刺失败，生成器会获得详细的反馈。

Before each sprint, the generator and evaluator negotiated a sprint contract: agreeing on what "done" looked like for that chunk of work before any code was written. This existed because the product spec was intentionally high-level, and I wanted a step to bridge the gap between user stories and testable implementation. The generator proposed what it would build and how success would be verified, and the evaluator reviewed that proposal to make sure the generator was building the right thing. The two iterated until they agreed.

每次冲刺前，生成器和评估者会协商冲刺合同：在编写代码前，就该部分工作的“完成”状态达成一致。之所以存在，是因为产品规格有意设计得很高级，我想做一个跨越用户故事和可测试实现之间的桥梁。发电机提出了它将建造什么以及如何验证成功方案，评估员会审查该提案以确保生成器建造的是正确的设备。两人反复尝试，直到达成一致。

Communication was handled via files: one agent would write a file, another agent would read it and respond either within that file or with a new file that the previous agent would read in turn. The generator then built against the agreed-upon contract before handing the work off to QA. This kept the work faithful to the spec without over-specifying implementation too early.

通信通过文件处理：一个代理写入文件，另一个代理读取并响应该文件内或前一个代理读取的新文件。发电机随后根据约定合同进行建设，随后将工作交给质量保证。这样既忠实于规范，又不会过早过早地规定实现。

Running the harness 运行安全带

For the first version of this harness, I used Claude Opus 4.5, running user prompts against both the full harness and a single-agent system for comparison. I used Opus 4.5 since this was our best coding model when I began these experiments.

在该框架的第一个版本中，我使用了 Claude Opus 4.5，并对完整框架和单代理系统运行用户提示以便对比。我用了 Opus 4.5，因为这是我开始做实验时最好的编码模型。

I wrote the following prompt to generate a retro video game maker:

我写了以下提示来生成一个复古电子游戏制作者：

Create a 2D retro game maker with features including a level editor, sprite editor, entity behaviors, and a playable test mode.

创建一个 2D 复古游戏制作器，具备关卡编辑器、精灵编辑器、实体行为和可玩测试模式等功能。

The table below shows the harness type, length it ran for, and the total cost.

下表显示了安全带类型、运行长度及总成本。

Harness 安全带

Duration 持续时间

Cost 费用

Solo 只有
20 min 20分钟
\$9

Full harness 全套安全带
6 hr 6小时
\$200

The harness was over 20x more expensive, but the difference in output quality was immediately apparent.

线束价格贵了20多倍，但输出质量的差异立刻显现出来。

I was expecting an interface where I could construct a level and its component parts (sprites, entities, tile layout) then hit play to actually play the level. I started by opening the solo run’s output, and the initial application seemed in line with those expectations.

我原本期待一个界面，可以构建关卡及其组成部分（精灵、实体、瓦片布局），然后点击播放键来实际游玩关卡。我先打开了单人跑的输出，最初的申请看起来符合这些预期。

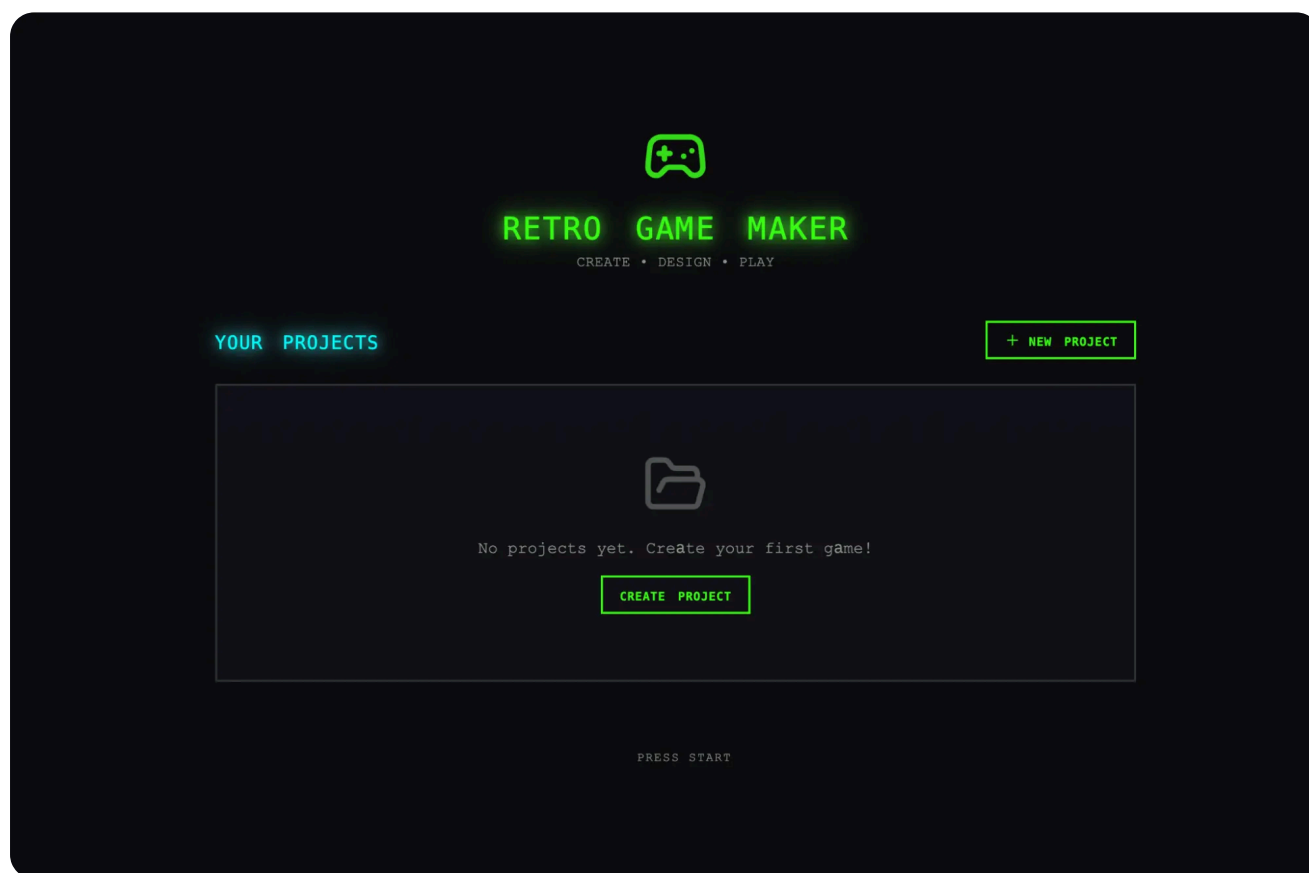
As I clicked through, however, issues started to emerge. The layout wasted space, with fixed-height panels leaving most of the viewport empty. The workflow was rigid. Trying to populate a level prompted me to create sprites and entities first, but nothing in the UI guided me toward that sequence. More to the point, the actual game was broken. My entities appeared on screen but nothing responded to input. Digging into the code revealed that the wiring between entity definitions and the game runtime was broken, with no surface indication of where.

然而，随着我点击，问题开始浮现。布局浪费了空间，固定高度的面板让大部分视窗空无一物。工作流程很僵硬。尝试填充关卡时，提示我先创建精灵和实体，但界面没有引导我进入那个顺序。更重要的是，游戏本身就有问题。我的实体出现在屏幕上，但输入没有任何回应。深入代码后发现实体定义与游戏运行时之间的连接断裂，且没有表面指示。

Opening screen 开场画面

Sprite editor 精灵编辑器

Game play 游戏玩法



Initial screen when opening the app created by the solo harness.

打开由 Solo 安全带创建的应用时的初始界面。

After evaluating the solo run, I turned my attention to the harness run. This run started from the same one-sentence prompt, but the planner step expanded that prompt into a 16-feature spec spread across ten sprints. It went well beyond what the solo run attempted. In addition to the core editors and play mode, the spec called for a sprite animation system, behavior templates, sound effects and music, an AI-assisted sprite generator and level designer, and game export with shareable links. I gave the planner access to our frontend design skill, which it read and used to create a visual design language for the app as part of the spec. For each sprint, the generator and evaluator negotiated a contract defining the specific implementation details for the sprint, and the testable behaviors that would be tested to verify completion.

评估完单人跑步后，我把注意力转向了背带跑步。这次运行从同一个一句话提示开始，但规划步骤将该提示扩展成 16 个功能，分布在十个冲刺中。它远远超出了单人连载的尝试。除了核心编辑器和游戏模式外，规范还要求精灵动画系统、行为模板、音效和音乐、AI 辅助的精灵生成器和关卡设计师，以及带可分享链接的游戏导出功能。我给规划师使用了我们的前端设计技能，它读取后用它为应用创建了视觉设计语言，作为规范的一部分。对于每个冲刺，生成器和评估器协商合同，定义该冲刺的具体实现细节以及用于验证完成的可测试行为。

The app immediately showed more polish and smoothness than the solo run. The canvas used the full viewport, the panels were sized sensibly, and the interface had a consistent visual identity that tracked the design direction from the spec. Some of the clunkiness I'd seen in the solo run did remain—the workflow still didn't make it clear that you should build sprites and entities before trying to populate a level, and I had to figure that out by poking around. This read as a gap in the base model's product intuition rather than something the harness was designed to address, though it did suggest a place where targeted iteration inside the harness

could help to further improve output quality.

应用立刻展现出比单人跑更精致、更顺畅的体验。画布采用了完整的视窗，面板尺寸合理，界面具有统一的视觉识别，紧跟着规格设计方向。我在单人游戏中看到的一些笨拙依然存在——工作流程仍然没有明确说明在尝试填充关卡前应该先构建精灵和实体，我只能自己摸索。这更像是基础模型产品直觉中的一个空白，而非线束设计的目的，尽管它暗示线束内部的有针对性迭代可以进一步提升输出质量。

Working through the editors, the new run's advantages over solo became more apparent. The sprite editor was richer and more fully featured, with cleaner tool palettes, a better color picker, and more usable zoom controls.

通过编辑的努力，新一轮相较于单人版的优势变得更加明显。精灵编辑器功能更丰富，工具调色板更简洁，颜色选择更佳，缩放控制更实用。

Because I'd asked the planner to weave AI features into its specs, the app also came with a built-in Claude integration that let me generate different parts of the game through prompting. This significantly sped up the workflow.

因为我让策划器把 AI 功能融入了它的规格中，应用还内置了 Claude 集成，让我可以通过提示生成游戏的不同部分。这大大加快了工作流程。

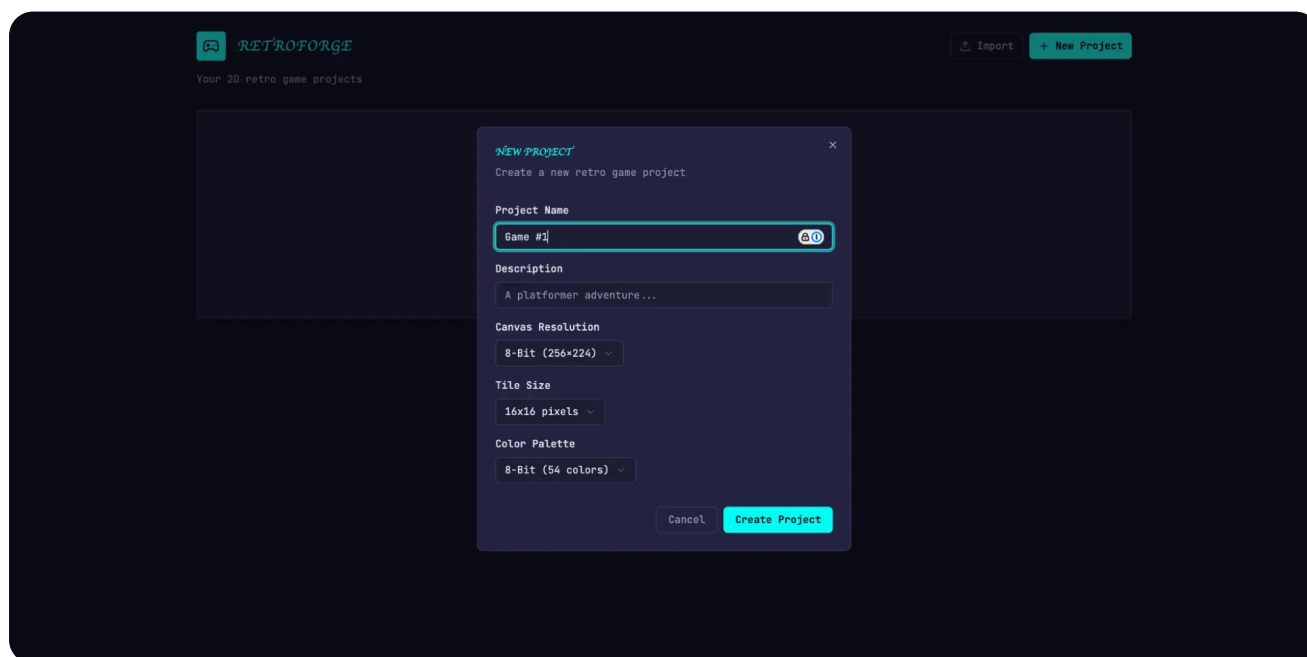
Opening screen 开场画面

Sprite editor 精灵编辑器

AI game design AI 游戏设计

AI game design AI 游戏设计

Game play 游戏玩法



Initial screen: Creating a new game, in the app built with the full harness

初始界面：在应用中创建新游戏，使用完整线束

The biggest difference was in play mode. I was actually able to move my entity and play the game. The physics had some rough edges—my character jumped onto a platform but ended up overlapping with it, which felt intuitively wrong—but the core thing worked, which the solo run did not manage. After moving around a bit, I did hit some limitations with the AI's game level construction. There was a large wall that I wasn't able to jump past, so I was stuck. This suggested there were some common sense improvements and edge cases that the harness could handle to further refine the app.

最大的区别是在游戏模式中。我实际上能够移动我的实体并继续游戏。物理效果有些粗糙——我的角色跳上了一个平台，但结果和平台重叠，感觉直觉上不太对劲——但核心机制是可行的，单人通关没做到这一点。在移动了一会儿后，我在 AI 的关卡构建上遇到了一些限制。有一堵大墙我跳不过去，所以被困住了。这表明安全带有一些常识性的改进和边缘情况，可以进一步完善应用。

Reading through the logs, it was clear that the evaluator kept the implementation in line with the spec. Each sprint, it walked through the sprint contract's test criteria and exercised the running application through Playwright, filing bugs against anything that diverged from expected behavior. The contracts were granular—Sprint 3 alone had 27 criteria covering the level editor—and the evaluator's findings were specific enough to act on without extra investigation. The

table below shows several examples of issues our evaluator identified:

仔细阅读日志，很明显评估者让实现符合规范。每次冲刺，它都会逐一完成冲刺合同的测试标准，并通过 Playwright 执行运行中的应用程序，对任何偏离预期行为的行为提交错误。合同非常细致——仅 Sprint 3 就有 27 项标准涵盖关卡编辑器——评估者的发现足够具体，无需额外调查即可采取行动。下表展示了评估员识别的几个问题示例：

Contract criterion 合同标准

Evaluator finding 评估器发现

Rectangle fill tool allows click-drag to fill a rectangular area with selected tile
矩形填充工具允许点击拖曳，用选中的瓷砖填充矩形区域

FAIL — Tool only places tiles at drag start/end points instead of filling the region.
`fillRectangle` function exists but isn't triggered properly on mouseUp.

失败 —— 工具只在拖动起点/终点放置瓦片，而不是填充该区域。
`fillRectangle` 函数存在，但在 mouseUp 上没有正确触发。

User can select and delete placed entity spawn points
用户可以选择并删除已放置的实体生成点

FAIL — Delete key handler at `LevelEditor.tsx:892` requires both `selection` and `selectedEntityId` to be set, but clicking an entity only sets `selectedEntityId`. Condition should be `selection || (selectedEntityId && activeLayer === 'entity')`.

FAIL — `LevelEditor.tsx: 892` 的删除键处理程序要求同时设置 `select` 和 `selectEntityId`，但点击实体只设置 `selectedEntityId`。条件应为 `selection || (selectedEntityId && activeLayer === 'entity')`。

User can reorder animation frames via API
用户可以通过 API 重新排序动画帧

FAIL — `PUT /frames/reorder` route defined after `/ {frame_id}` routes. FastAPI matches 'r eorder' as a frame_id integer and returns 422: "unable to parse string as an integer."

失败 — 在 `/ {frame_id}` 路由后定义的 `/frames/reorder` 路由。FastAPI 匹配 'r eorder' 为 frame_id 整数，返回 422：“无法解析字符串为整数。”

Getting the evaluator to perform at this level took work. Out of the box, Claude is a poor QA agent. In early runs, I watched it identify legitimate issues, then talk itself into deciding they weren't a big deal and approve the work anyway. It also tended to test superficially, rather than probing edge cases, so more subtle bugs often slipped through. The tuning loop was to read the evaluator's logs, find examples where its judgment diverged from mine, and update the QAs prompt to solve for those issues. It took several rounds of this development loop before the evaluator was grading in a way that I found reasonable. Even then, the harness output showed the limits of the model's QAing capabilities: small layout issues, interactions that felt unintuitive in places, and undiscovered bugs in more deeply nested features that the evaluator hadn't exercised thoroughly. There was clearly more verification headroom to capture with further tuning. But compared to the solo run, where the central feature of the application simply didn't work, the lift was obvious.

让评估员达到这个水平需要付出努力。Claude 从一开始就是个糟糕的质量保证员。早期运行时，我看到它识别出合理问题，然后自己说服自己决定这些问题不算什么，最终还是批准了工作。它也倾向于表面测试，而非深入探究边缘情况，因此更细微的漏洞常常会被漏网。调整循环是阅读评估者的日志，找出其判断与我意见不符的例子，并更新质量保证提示以解决这些问题。经过几轮这样的开发循环，评估员才以我认为合理的方式评分。即便如此，线束输出也暴露了模型质量保证能力的局限：小的布局问题，部分操作感觉不直观，以及评估者未彻底处理的更深嵌套功能中未发现的漏洞。显然，进一步调校后还有更多的验证余量。但相比单人运行，应用的核心功能根本无法起作用，提升的效果显而易见。

Iterating on the harness

对线束的改进

The first set of harness results was encouraging, but it was also bulky, slow, and expensive. The logical next step was to find ways to simplify the harness without degrading its performance. This was partly common sense and partly a function of a more general principle: every component in a harness encodes an assumption about what the model can't do on its own, and those assumptions are worth stress testing, both because they may be incorrect, and because they can quickly go stale as models improve. Our blog post [Building Effective Agents](#) frames the underlying idea as "find the simplest solution possible, and only increase complexity when needed," and it's a pattern that shows up consistently for anyone maintaining an

agent harness.

第一组背带的结果令人鼓舞，但体积庞大、速度慢且价格昂贵。下一步合乎逻辑的是寻找简化线束的方法，同时不降低其性能。这部分是常识，部分是更普遍的原则：线束中的每个组件都编码了模型自身无法完成的假设，这些假设值得进行压力测试，因为它们可能错误，也因为随着模型改进，这些假设可能很快变得乏味。我们的博客文章 [《打造高效的代理》](#) 将核心理念框架为“找到最简单的解决方案，仅在必要时增加复杂度”，这一模式在维护代理工具束的人们中始终如一。

In my first attempt to simplify, I cut the harness back radically and tried a few creative new ideas, but I wasn't able to replicate the performance of the original. It also became difficult to tell which pieces of the harness design were actually load-bearing, and in what ways. Based on that experience, I moved to a more methodical approach, removing one component at a time and reviewing what impact it had on the final result.

在我第一次尝试简化时，我大幅削减了背带，尝试了一些创意新点子，但没能复制原版的性能。也变得难以判断安全带设计中哪些部件是承重的，以及以何种方式承载。基于这些经验，我采取了更为系统化的方法，逐个移除一个组件，并回顾它对最终结果的影响。

As I was going through these iteration cycles, we also released Opus 4.6, which provided further motivation to reduce harness complexity. There was good reason to expect 4.6 would need less scaffolding than 4.5 did. From our [launch blog](#): "[Opus 4.6] plans more carefully, sustains agentic tasks for longer, can operate more reliably in larger codebases, and has better code review and debugging skills to catch its own mistakes." It also improved substantially on long-context retrieval. These were all capabilities the harness had been built to supplement.

在经历这些迭代周期的同时，我们也发布了 Opus 4.6，进一步激励了降低线束复杂度。有充分理由预期 4.6 版所需的支架会比 4.5 版少。摘自我们的[发布博客](#)：“[Opus 4.6]规划更周密，能更长时间地维持代理任务，能更可靠地在更大代码库中运行，并且具备更好的代码审查和调试能力，能够发现自身的错误。”它在长上下文检索方面也有了显著提升。这些都是背带设计的补充能力。

Removing the sprint construct 移除冲刺构造

I started by removing the sprint construct entirely. The sprint structure had helped to decompose work into chunks for the model to work coherently. Given the improvements in Opus 4.6, there was good reason to believe that the model could

natively handle the job without this sort of decomposition.

我一开始完全移除了冲刺结构。冲刺结构帮助将工作分解成多个块，使模型能够连贯地工作。鉴于 Opus 4.6 的改进，有充分理由相信该模型能够在不出现这种分解的情况下原生处理该任务。

I kept both the planner and evaluator, as each continued to add obvious value.

Without the planner, the generator under-scoped: given the raw prompt, it would start building without first spec'ing its work, and end up creating a less feature-rich application than the planner did.

我保留了规划器和评估者，因为他们各自都带来了明显的价值。没有规划器，生成器的范围被低估了：根据原始提示，它会在未先明确工作规格的情况下开始构建，最终生成的功能不如规划器丰富。

With the sprint construct removed, I moved the evaluator to a single pass at the end of the run rather than grading per sprint. Since the model was much more capable, it changed how load-bearing the evaluator was for certain runs, with its usefulness depending on where the task sat relative to what the model could do reliably on its own. On 4.5, that boundary was close: our builds were at the edge of what the generator could do well solo, and the evaluator caught meaningful issues across the build. On 4.6, the model's raw capability increased, so the boundary moved outward. Tasks that used to need the evaluator's check to be implemented coherently were now often within what the generator handled well on its own, and for tasks within that boundary, the evaluator became unnecessary overhead. But for the parts of the build that were still at the edge of the generator's capabilities, the evaluator continued to give real lift.

移除冲刺结构后，我把评估器改为跑完时的单次评分，而不是每次冲刺评分。由于模型能力更强，它改变了评估器在某些运行中的承重程度，其实用性取决于任务的位置与模型自身的可靠性能力。在4.5版本中，这个界限很接近：我们的构建已经接近生成器单人能做的极限，评估器发现了整个构建中的重要问题。在4.6版本中，模型的原始能力提升，边界向外移动。那些原本需要评估员检查才能连贯实现的任务，现在往往在生成器自身能很好地处理的范围内，而在该范围内的任务，评估者就变成了不必要的开销。但对于仍在发电机能力边缘的部分，评估器依然给予了真正的提升。

The practical implication is that the evaluator is not a fixed yes-or-no decision. It is worth the cost when the task sits beyond what the current model does reliably solo.

实际意义在于，评估者并非固定的“是”或“否”决定。当这项任务超出现有型号单人可靠能力时，这笔费用是值得的。

Alongside the structural simplification, I also added prompting to improve how the harness built AI features into each app, specifically getting the generator to build a proper agent that could drive the app's own functionality through tools. That took real iteration, since the relevant knowledge is recent enough that Claude's training data covers it thinly. But with enough tuning, the generator was building agents correctly.

除了结构简化，我还加入了提示，改进了 Harness 在每个应用中内置 AI 功能的方式，特别是让生成器构建一个合适的代理，通过工具驱动应用自身的功能。这需要大量迭代，因为相关知识足够新，克劳德的训练数据对此仅有较少覆盖。但只要调校足够，生成器就能正确构建代理。

Results from the updated harness

更新后的安全带结果

To put the updated harness to the test, I used the following prompt to generate a Digital Audio Workstation (DAW), a music production program for composing, recording, and mixing songs:

为了测试更新后的线束，我使用以下提示生成了一个数字音频工作站（DAW），这是一个用于作曲、录音和混音的音乐制作程序：

Build a fully featured DAW in the browser using the Web Audio API.

在浏览器中使用 Web Audio API 构建一个功能齐全的 DAW。

The run was still lengthy and expensive, at about 4 hours and \$124 in token costs.

这趟演出依然漫长且费用高昂，约4小时，象征性费用为124美元。

Most of the time went to the builder, which ran coherently for over two hours without the sprint decomposition that Opus 4.5 had needed.

大部分时间都交给了构建者，它连续运行了两个多小时，没有 Opus 4.5 所需的冲刺分解。

Agent & Phase 代理与相位

Duration 持续时间

Cost 费用

Planner 规划师
4.7 min 4.7分钟
\$0.46

Build (Round 1) 构建（第一轮）
2 hr 7 min 2小时7分钟
\$71.08

QA (Round 1) QA（第一轮）
8.8 min 8.8分钟
\$3.24

Build (Round 2) 构建（第二轮）
1 hr 2 min 1小时2分钟
\$36.89

QA (Round 2) QA（第二轮）
6.8 min 6.8分钟
\$3.09

Build (Round 3) 构建（第三轮）
10.9 min 10.9分钟
\$5.88

QA (Round 3) 质量保证（第三轮）
9.6 min 9.6分钟
\$4.06

Total V2 Harness Total V2 安全带
3 hr 50 min 3小时50分钟
\$124.70

As with the previous harness, the planner expanded the one-line prompt into a full spec. From the logs, I could see the generator model did a good job planning the

app and the agent design, wiring the agent up, and testing it before handing off to QA.

和之前的线索一样，规划师将单行提示扩展为完整的规格。从日志来看，生成器模型在规划应用和代理设计方面做得很好，代理布线并测试后才交给质量保证。

That being said, the QA agent still caught real gaps. In its first-round feedback, it noted:

话虽如此，质量保证人员还是发现了真正的漏洞。在第一轮反馈中，它指出：

This is a strong app with excellent design fidelity, solid AI agent, and good backend. The main failure point is Feature Completeness — while the app looks impressive and the AI integration works well, several core DAW features are display-only without interactive depth: clips can't be dragged/moved on the timeline, there are no instrument UI panels (synth knobs, drum pads), and no visual effect editors (EQ curves, compressor meters). These aren't edge cases — they're the core interactions that make a DAW usable, and the spec explicitly calls for them.

这是一个设计精良、AI 代理扎实、后端功能强劲的强大应用。主要的缺陷是功能完整性——虽然应用看起来很出色，AI 集成也很好，但有几个核心 DAW 功能仅显示，没有互动深度：无法在时间线上拖动或移动片段，没有乐器界面面板（合成旋钮、鼓垫），也没有视觉效果编辑器（均衡曲线、压缩器计量）。这些不是极端情况——它们是让 DAW 可用的核心交互，规范明确要求使用它们。

In its second round feedback, it again caught several functionality gaps:

在第二轮反馈中，它再次发现了几个功能上的漏洞：

Remaining gaps: 剩余的空白：

- Audio recording is still stub-only (button toggles but no mic capture)
- 音频录制仍为存根（按钮切换但无麦克风捕捉）
- Clip resize by edge drag and clip split not implemented
- 通过边缘拖拽和剪辑分割调整剪辑大小，未实现
- Effect visualizations are numeric sliders, not graphical (no EQ curve)
- 效果可视化是数字滑块，而非图形化（无均衡曲线）

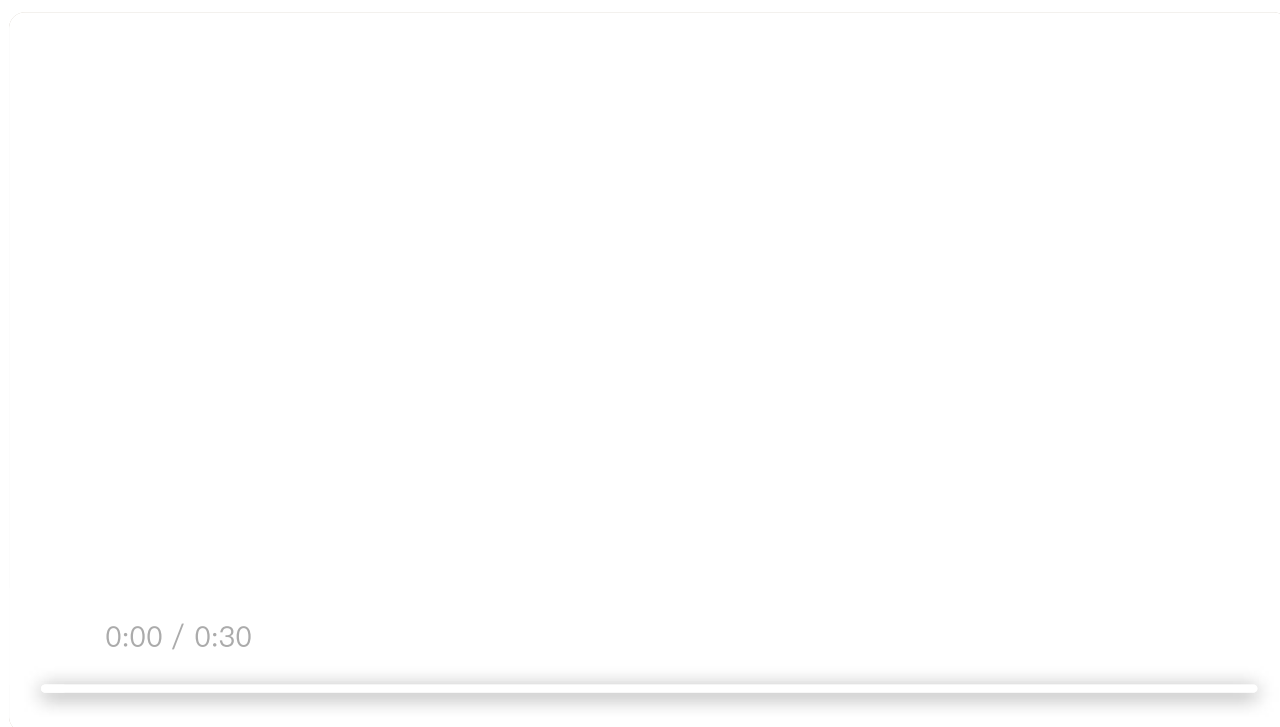
The generator was still liable to miss details or stub features when left to its own devices, and the QA still added value in catching those last mile issues for the

generator to fix.

生成器在自行处理时仍可能遗漏细节或存根功能，而质量保证仍能为生成器发现最后一公里的问题提供帮助。

Based on the prompt, I was expecting a program where I could create melodies, harmonies, and drum patterns, arrange them into a song, and get help from an integrated agent along the way. The video below shows the result.

根据这个提示，我原本期待一个能创作旋律、和声和鼓点、将它们编排成一首歌，并在过程中获得集成代理帮助的程序。下面的视频展示了结果。



The app is far from a professional music production program, and the agent's song composition skills could clearly use a lot of work. Additionally, Claude can't actually hear, which made the QA feedback loop less effective with respect to musical taste.

这个应用远非专业的音乐制作项目，代理人的歌曲创作能力显然还有很大提升空间。此外，Claude 实际上听不到声音，这使得 QA 反馈环节在音乐品味上的效果降低。

But the final app had all the core pieces of a functional music production program: a working arrangement view, mixer, and transport running in the browser. Beyond that, I was able to put together a short song snippet entirely through prompting: the agent set the tempo and key, laid down a melody, built a drum track, adjusted mixer levels, and added reverb. The core primitives for song composition were present, and the agent could drive them autonomously, using tools to create a simple production from end to end. You might say it's not pitch-perfect yet—but it's getting there.

但最终的应用包含了功能性音乐制作程序的所有核心要素：浏览器中运行的工作编曲视图、混音器和传输。除此之外，我完全靠提示拼凑出了一段短歌片段：经纪人设定了节奏和调性，录了旋律，制作了鼓轨，调整了混音器音量，并添加了混响。歌曲创作的核心原语已具备，代理可以自主驱动这些原语，利用工具从头到尾创建简单的制作过程。你可能会说还没完全完美——但已经在接近了。

What comes next 接下来会发生什么

As models continue to improve, we can roughly expect them to be capable of working for longer, and on more complex tasks. In some cases, that will mean the scaffold surrounding the model matters less over time, and developers can wait for the next model and see certain problems solve themselves. On the other hand, the better the models get, the more space there is to develop harnesses that can achieve complex tasks beyond what the model can do at baseline.

随着模型的不断改进，我们可以大致预期它们能够工作更长时间，并完成更复杂的任务。在某些情况下，这意味着模型周围的支架随着时间推移变得不那么重要，开发者可以等待下一个模型，看到某些问题自行解决。另一方面，模型越优秀，开发能够完成复杂任务的束缚空间就越大，超出模型基线能力。

With this in mind, there are a few lessons from this work worth carrying forward. It is always good practice to experiment with the model you're building against, read its traces on realistic problems, and tune its performance to achieve your desired outcomes. When working on more complex tasks, there is sometimes headroom from decomposing the task and applying specialized agents to each aspect of the problem. And when a new model lands, it is generally good practice to re-examine a harness, stripping away pieces that are no longer load-bearing to performance and adding new pieces to achieve greater capability that may not have been

possible before.

考虑到这一点，这项工作中有一些值得继续传承的教训。在你构建的模型上进行实验，阅读其在现实问题上的痕迹，并调整其性能以实现理想结果，总是很好的做法。在处理更复杂的任务时，有时还能从分解任务并对问题的各个方面应用专门的代理来实现。当新型号降落时，通常建议重新检查背带，拆除不再承重的部件，增加新部件以实现以前可能无法实现的性能。

From this work, my conviction is that the space of interesting harness combinations doesn't shrink as models improve. Instead, it moves, and the interesting work for AI engineers is to keep finding the next novel combination.

通过这项工作，我坚信有趣的束缚组合空间不会随着模型的改进而缩小。相反，它在移动，而 AI 工程师有趣的工作是不断寻找下一个新颖的组合。

Acknowledgements 致谢

Special thanks to Mike Krieger, Michael Agaby, Justin Young, Jeremy Hadfield, David Hershey, Julius Tarn, Xiaoyi Zhang, Barry Zhang, Orowa Sidker, Michael Tingley, Ibrahim Madha, Martina Long, and Canyon Robbins for their contributions to this work.

特别感谢迈克·克里格、迈克尔·阿加比、贾斯汀·杨、杰里米·哈德菲尔德、大卫·赫希、朱利叶斯·坦、张晓怡、张巴里、奥罗瓦·西德克、迈克尔·廷格利、易卜拉欣·马达、玛蒂娜·朗和峡谷·罗宾斯对本工作的贡献。

Thanks also to Jake Eaton, Alyssa Leonard, and Stef Sequeira for their help shaping the post.

还要感谢 Jake Eaton、Alyssa Leonard 和 Stef Sequeira 为这篇文章提供帮助。

Appendix 附录

Example plan generated by planner agent.

由规划代理人生成的示例计划。

RetroForge - 2D Retro Game Maker

Overview

RetroForge is a web-based creative studio for designing and building 2D retro-style video games. It combines the nostalgic charm of classic 8-bit and 16-bit game aesthetics with modern, intuitive editing tools—enabling anyone from hobbyist creators to indie developers to bring their game ideas to life without writing traditional code.

The platform provides four integrated creative modules: a tile-based Level Editor for designing game worlds, a pixel-art Sprite Editor for crafting visual assets, a visual Entity Behavior system for defining game logic, and an instant Playable Test Mode for

 Copy 收到  Expand 扩展

Get the developer newsletter

订阅开发者通讯

Product updates, how-tos, community spotlights, and more.
Delivered monthly to your inbox.

产品更新、操作指南、社区聚焦灯光，以及更多内容。每月发送至您的邮箱。

Enter your email 

Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.

如果你想收到我们的月度开发者通讯，请提供你的电子邮箱。您可以随时取消订阅。



Products

Claude

Claude Code

Claude Code Enterprise

Claude Cowork

Claude for Chrome

Claude for Excel

Claude for PowerPoint

Claude for Slack

Skills

Max plan

Team plan

Enterprise plan

Download app

Pricing

Log in to Claude

Models

Opus

Sonnet

Haiku

Solutions

AI agents

Claude Code Security

Code modernization

Coding

Customer support

Education

Financial services

Government

Healthcare

Life sciences

Nonprofits

Claude Platform

Overview

Developer docs

Pricing

Marketplace

Regional compliance

Amazon Bedrock

Google Cloud's Vertex AI

Microsoft Foundry

Console login

Resources

Blog

Claude partner network

Community

Connectors

Courses

Customer stories

Engineering at Anthropic

Events

Inside Cowork

Plugins

Powered by Claude

Service partners

Startups program

Tutorials

Use cases

Company

Anthropic

Careers

Economic Futures

Research

News

Claude's Constitution

Responsible Scaling Policy

Security and compliance

Transparency

Help and security

Availability

Status

Support center

Terms and policies

Privacy choices

Privacy policy

Consumer health data privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

© 2026 Anthropic PBC

